Cheat Sheet: Retrieval-Augmented Generation (RAG) with Hugging Face and PyTorch

Listen to reading 8 minutes

Refer to this cheat sheet to learn more about various components/methods, their descriptions, and code examples.

tsne_plot(data)

This function applies t-SNE to reduce high-dimensional data such as embeddings into three dimensions and visualizes them using a 3D scatter plot. Each point represents an input sample, colored and labeled by its original order, allowing us to observe similarity, clustering, and relative relationships between data points.

Code Example:

```
    # Plot scatter with unique colors for each point
    for idx, point in enumerate(data_3d):
        ax.scatter(point[0], point[1], point[2], label=str(idx), color=colors[idx])
    # Adding labels and titles
    ax.set_xlabel('TSNE Component 1')
    ax.set_ylabel('TSNE Component 2')
    ax.set_zlabel('TSNE Component 3')
    plt.title('3D t-SNE Visualization')
    plt.legend(title='Input Order')
    plt.show()
```

read_and_split_text(filename)

This code reads a text file, splits its content into non-empty paragraphs based on line breaks, and returns them as a cleaned list for further processing.

Code Example:

```
def read_and_split_text(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        text = file.read()
    # Split the text into paragraphs (simple split by newline characters)
    paragraphs = text.split('\n')
    # Filter out any empty paragraphs or undesired entries
    paragraphs = [para.strip() for para in paragraphs if len(para.strip()) > 0]
    return paragraphs
# Read the text file and split it into paragraphs
paragraphs = read_and_split_text('companyPolicies.txt')
paragraphs[0:10]
```

DPRContextEncoderTokenizer

DPRContextEncoderTokenizer is functionally identical to BertTokenizer and performs end-to-end text tokenization by first splitting text into words and punctuation, and then further breaking words into subword units using the WordPiece algorithm.

Code Example:

```
%%capture
```

```
context_tokenizer = DPRContextEncoderTokenizer.from_pretrained('facebook/dpr-ctx_encoder-si
context_tokenizer
```

encode_contexts(text_list)

This code tokenizes a list of text paragraphs, encodes them using a DPR context encoder to generate fixed-size embeddings, and combines them into a NumPy array for downstream similarity or retrieval tasks.

Code Example:

```
def encode_contexts(text_list):
    # Encode a list of texts into embeddings
    embeddings = []
    for text in text_list:
        inputs = context_tokenizer(text, return_tensors='pt', padding=True, truncation=True,
        outputs = context_encoder(**inputs)
        embeddings.append(outputs.pooler_output)
    return torch.cat(embeddings).detach().numpy()
# you would now encode these paragraphs to create embeddings.
context_embeddings = encode_contexts(paragraphs)
```

IndexFlatL2

IndexFlatL2 is one of the simplest and most used indexes in FAISS. It computes the Euclidean distance (L2 norm) between the query vector and the dataset vectors to determine similarity. This method is straightforward but very effective for many use cases where the exact distance calculation is crucial.

Code Example:

```
import faiss
# Convert list of numpy arrays into a single numpy array
embedding_dim = 768  # This should match the dimension of your embeddings
context_embeddings_np = np.array(context_embeddings).astype('float32')
# Create a FAISS index for the embeddings
index = faiss.IndexFlatL2(embedding_dim)
index.add(context_embeddings_np)  # Add the context embeddings to the index
```

search_relevant_contexts(question, question_tokenizer, question_encoder, index, k=5)

This function converts a question into an embedding using a DPR question encoder and retrieves the top-k most relevant context embeddings from an index based on similarity search.

Code Example:

```
def search_relevant_contexts(question, question_tokenizer, question_encoder, index, k=5):
    """
    Searches for the most relevant contexts to a given question.
    Returns:
    tuple: Distances and indices of the top k relevant contexts.
    """
    # Tokenize the question
    question_inputs = question_tokenizer(question, return_tensors='pt')
    # Encode the question to get the embedding
    question_embedding = question_encoder(**question_inputs).pooler_output.detach().numpy()
    # Search the index to retrieve top k relevant contexts
    D, I = index.search(question_embedding, k)
    return D, I
```

GPT2 model and tokenizer

This code is a combination of GPT2 for generation and DPR for question encoding creates a robust framework for your natural language processing application, enabling it to deliver accurate and context-aware responses to user inquiries.

Code Example:

```
tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")
model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
model.generation_config.pad_token_id = tokenizer.pad_token_id
```

generate_answer_without_context(question)

This function generates an answer directly from a question by tokenizing it and using a pretrained language model to produce a text response without relying on any external context.

Code Example:

```
def generate_answer_without_context(question):
    # Tokenize the input question
    inputs = tokenizer(question, return_tensors='pt', max_length=1024, truncation=True)
    # Generate output directly from the question without additional context
    summary_ids = model.generate(inputs['input_ids'], max_length=150, min_length=40, length_
                                 num_beams=4, early_stopping=True,pad_token_id=tokenizer.eos
    # Decode and return the generated text
    answer = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
    return answer
```

generate_answer(question, contexts)

This function generates an answer by combining the question with retrieved context passages, tokenizing the combined input, and using a language model to produce a context-aware response.

Code Example:

```
def generate_answer(question, contexts):
    # Concatenate the retrieved contexts to form the input to GPT2
    input_text = question + ' ' + ' '.join(contexts)
    inputs = tokenizer(input_text, return_tensors='pt', max_length=1024, truncation=True)
    # Generate output using GPT2
    summary_ids = model.generate(inputs['input_ids'], max_new_tokens=50, min_length=40, leng
                                 num_beams=4, early_stopping=True,pad_token_id=tokenizer.eos
    return tokenizer.decode(summary_ids[0], skip_special_tokens=True)
```

BertTokenizer

This code imports the BERT tokenizer and loads the pretrained bert-base-uncased tokenizer to convert text into lowercase subword tokens that the BERT model can understand.

Code Example:

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

Loading the BERT model

This code loads the pretrained bert-base-uncased BERT model to generate contextual embeddings for input text.

Code Example:

```
from transformers import BertModel
bert_model = BertModel.from_pretrained('bert-base-uncased')
```

aggregated_mean_embeddings

This code generates a single mean BERT embedding per input sequence by removing padding tokens using the attention mask and averaging the remaining word embeddings.

Code Example:

```
# Initialize a list to store the mean embeddings for each input sequence
aggregated_mean_embeddings = []
# Loop over each pair of input_ids and attention_masks
for token_ids, attention_mask in tqdm(zip(input_ids['input_ids'], input_ids['attention_mask
    # Convert list of token ids and attention mask to tensors
    token_ids_tensor = torch.tensor([token_ids]).to(DEVICE)
    attention_mask_tensor = torch.tensor([attention_mask]).to(DEVICE)
    print("token_ids_tensor shape:",token_ids_tensor.shape, attention_mask_tensor.shape)  #
    with torch.no_grad():  # Disable gradient calculations for faster execution
        # Retrieve the batch of word embeddings from the BERT model
        embeddings = bert_model(token_ids_tensor, attention_mask=attention_mask_tensor)[0].s
        print("Word embeddings shape:", embeddings.shape)
        # Count and print the number of zero-padding embeddings
        num_zero_paddings = (attention_mask_tensor == 0).sum().item()
        print("Number of zero-padding embeddings:", num_zero_paddings)
```

```
        print("Number of zero padding embeddings:", num_zero_paddings)
        # Create a mask for positions that are not zero-padded
        valid_embeddings_mask = attention_mask_tensor[0] != 0
        print("valid_embeddings_mask:",valid_embeddings_mask)
        # Filter out the embeddings corresponding to zero-padded positions
        filtered_embeddings = embeddings[valid_embeddings_mask, :]
        print("Word embeddings after zero padding embeddings removed:", filtered_embeddings
        # Compute the mean of the filtered embeddings
        mean_embedding = filtered_embeddings.mean(axis=0)
        print("Mean embedding shape:", mean_embedding.shape)
        # Append the mean embedding to the list, adding a batch dimension
aggregated_mean_embeddings.append(mean_embedding.unsqueeze(0))
# Concatenate all mean embeddings to form a single tensor
aggregated_mean_embeddings = torch.cat(aggregated_mean_embeddings)
print('All mean embeddings shape:', aggregated_mean_embeddings.shape)
```

aggregate_embeddings(input_ids, attention_masks, bert_model=bert_model)

This function passes each input sequence through BERT to obtain token-level embeddings, removes embeddings corresponding to padded tokens using the attention mask, and computes the mean of the valid embeddings. The result is a single fixed-size embedding vector representing each input text.

Code Example:

```
def aggregate_embeddings(input_ids, attention_masks, bert_model=bert_model):
    """
    Converts token indices and masks to word embeddings, filters out zero-padded embeddings,
    and aggregates them by computing the mean embedding for each input sequence.
    """
    mean_embeddings = []
    # Process each sequence in the batch
    print('number of inputs',len(input_ids))
    for input_id, mask in tqdm(zip(input_ids, attention_masks)):
        input_ids_tensor = torch.tensor([input_id]).to(DEVICE)
        mask_tensor = torch.tensor([mask]).to(DEVICE)
        with torch.no_grad():
            # Obtain the word embeddings from the BERT model
            word_embeddings = bert_model(input_ids_tensor, attention_mask=mask_tensor)[0].s
            # Filter out the embeddings at positions where the mask is zero
            valid_embeddings_mask=mask_tensor[0] != 0
            valid_embeddings = word_embeddings[valid_embeddings_mask,:]
            # Compute the mean of the filtered embeddings
            mean_embedding = valid_embeddings.mean(dim=0)
            mean_embeddings.append(mean_embedding.unsqueeze(0))
    # Concatenate the mean embeddings from all sequences in the batch
    aggregated_mean_embeddings = torch.cat(mean_embeddings)
    return aggregated_mean_embeddings
```

text_to_emb(list_of_text,max_input=512)

This function tokenizes a list of text inputs with padding and truncation, then converts them into fixed-size vector embeddings by passing the token IDs and attention masks through BERT and averaging the valid token embeddings.

Code Example:

```
def text_to_emb(list_of_text,max_input=512):
    data_token_index  = tokenizer.batch_encode_plus(list_of_text, add_special_tokens=True,pa
    return question_embeddings
```

RAG_QA(embeddings_questions, embeddings, n_responses=3)

This function performs retrieval in a RAG setup by computing similarity (dot product) between question embeddings and stored embeddings, ranking them by highest similarity, and printing the top-n_responses most relevant answers.

Code Example:

```
def RAG_QA(embeddings_questions, embeddings, n_responses=3):
    # Calculate the dot product between the question embeddings and the provided embeddings
    dot_product = embeddings_questions @ embeddings.T
    # Reshape the dot product results to a 1D tensor for easier processing.
    dot_product = dot_product.reshape(-1)
    # Sort the indices of the dot product results in descending order (setting descending t
    sorted_indices = torch.argsort(dot_product, descending=True)
    # Convert sorted indices to a list for easier iteration.
    sorted_indices = sorted_indices.tolist()
    # Print the top 'n_responses' responses from the sorted list, which correspond to the h
    for index in sorted_indices[:n_responses]:
        print(yes_responses[index])
```

Key Concepts

- **RAG (Retrieval-Augmented Generation)**: Architecture that enhances LLM responses by retrieving relevant external knowledge before generating answers. Combines information retrieval with text generation for contextually-aware responses.
- **Embeddings**: Dense vector representations of text that capture semantic meaning. BERT produces 768-dimensional vectors, MiniLM produces 384-dimensional vectors, enabling similarity-based retrieval.
- **BERT (Bidirectional Encoder Representations from Transformers)**: Pre-trained transformer model that generates contextual embeddings by processing text bidirectionally. Base model produces 768-dimensional vectors for semantic understanding.
- **DPR (Dense Passage Retriever)**: BERT-based model specialized for retrieval tasks. Uses separate encoders for questions and contexts, optimized through contrastive learning for superior passage retrieval performance.
- **Context Encoder**: Component of DPR that converts documents/passages into dense embeddings optimized for retrieval. Uses DPRContextEncoderTokenizer identical to BertTokenizer for text processing.
- **Question Encoder**: Component of DPR that converts queries into embeddings optimized to match with context embeddings. Enables asymmetric encoding where questions and passages are encoded differently.
- **Tokenization**: Process of converting text into tokens (subword units) that models can process. Includes special tokens like [CLS] (classification) and [SEP] (separator/end of sequence).
- **Attention Mask**: Binary tensor indicating which tokens are real content (1) vs padding (0). Essential for filtering padding tokens before computing mean embeddings.
- **Mean Pooling**: Aggregation strategy that converts variable-length token sequences into fixed-size vectors by averaging valid token embeddings (excluding padding). Creates single representative vector per document.
- **FAISS (Facebook AI Similarity Search)**: Efficient library for similarity search and clustering of dense vectors. Supports millions of vectors with sub-linear search time using approximate nearest neighbor algorithms.
- **IndexFlatL2**: FAISS index type that computes Euclidean distance (L2 norm) between query and dataset vectors for exact similarity search. Straightforward and effective for many retrieval use cases.
- **Vector Store**: Database optimized for storing and searching high-dimensional vectors using similarity metrics. FAISS is the most common implementation for RAG systems.
- **Dot Product Similarity**: Similarity metric computed as the dot product of two vectors. Higher values indicate greater similarity. Used for ranking retrieved documents in RAG.